

Padrões de Projeto (Builder)

QXD0068 - Reuso de Software

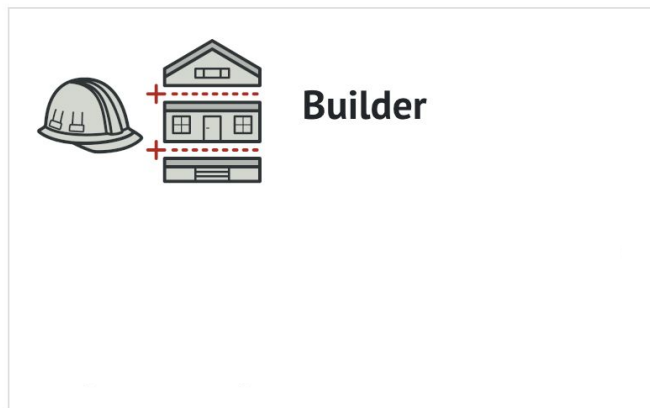
Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Builder
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação

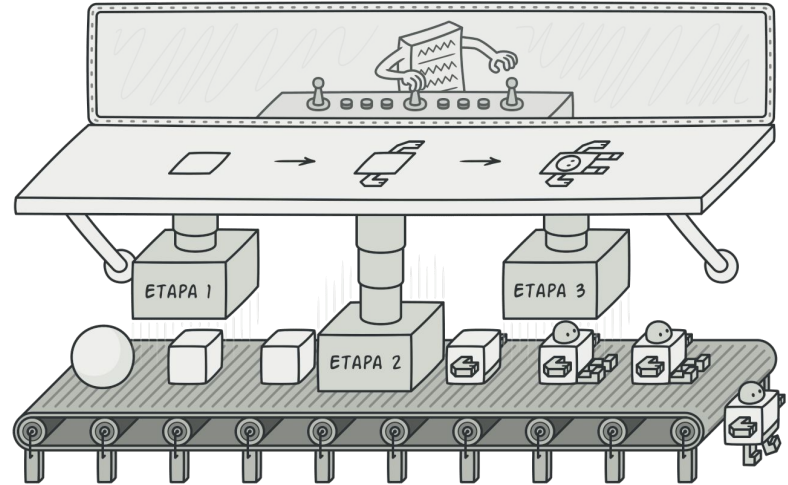


Introdução

- O Builder (Construtor) é um padrão de projeto criacional que permite criar objetos complexos passo a passo.
- Ele separa o processo de construção da representação final do objeto, possibilitando diferentes versões e combinações a partir do mesmo processo.
- Esse padrão é muito usado quando o construtor de uma classe se torna grande, confuso ou com muitos parâmetros opcionais.

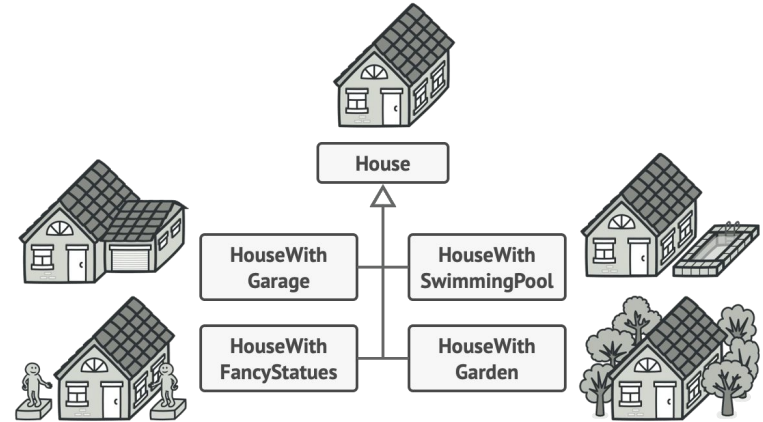
Padrão Builder

- O Builder define um objeto auxiliar (“builder”) responsável por montar um produto complexo em etapas controladas.
- Dessa forma, o mesmo código pode produzir diferentes representações de um produto — por exemplo, carros esportivos, SUVs ou manuais de instruções, todos criados a partir da mesma sequência de montagem.



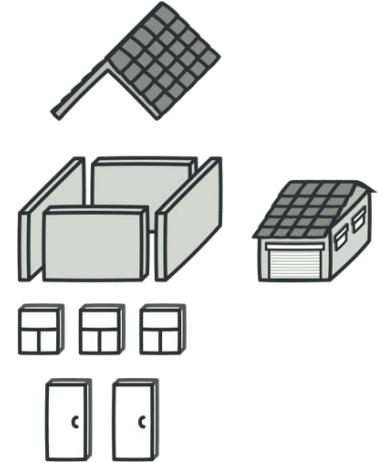
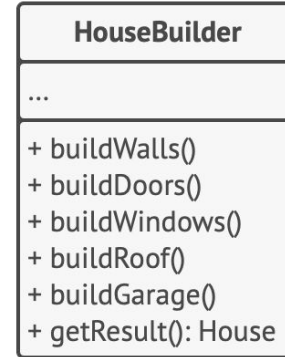
Problema

- Quando um objeto precisa ser construído com muitos parâmetros opcionais ou possui processo de inicialização extenso, surgem dois problemas clássicos:
 - Construtor telescópico – construtores com listas enormes de parâmetros.
 - Hierarquias complexas de subclasses – criadas para representar combinações de configurações.
- Essas soluções aumentam o acoplamento e a complexidade, tornando o código difícil de entender, estender e manter.



Solução

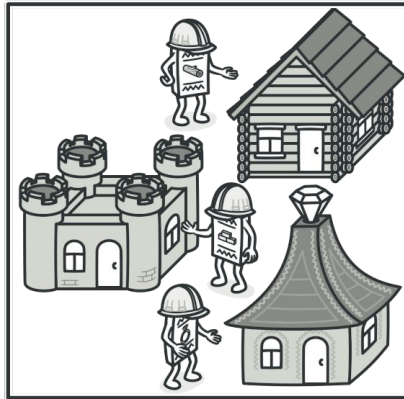
- O Builder propõe separar a lógica de construção do produto da sua estrutura.
- O código de montagem é movido para uma classe “builder” dedicada, que executa etapas bem definidas (como `setPortas()`, `setMotor()`, `setGPS()`).
- Além disso, um Diretor (Director) pode coordenar a sequência dessas etapas, garantindo consistência.
- Assim:
 - Cada Builder concreto cria uma variação específica do produto.
 -
 - O Diretor sabe “como montar”, mas não “o que está sendo montado”.



Analogia do Mundo Real

- Imagine um restaurante de fast-food:
 - O Atendente (Diretor) recebe o pedido e sabe a sequência para montar um lanche.
 - O Cozinheiro (Builder) prepara cada parte — pão, carne, salada, molho.
 - O Cliente (Código cliente) escolhe o tipo de lanche desejado.
 - Usando a mesma receita (ordem de montagem), é possível criar diferentes produtos: hambúrguer simples, duplo ou vegetariano — bastando mudar o Builder.

Analogia do Mundo Real



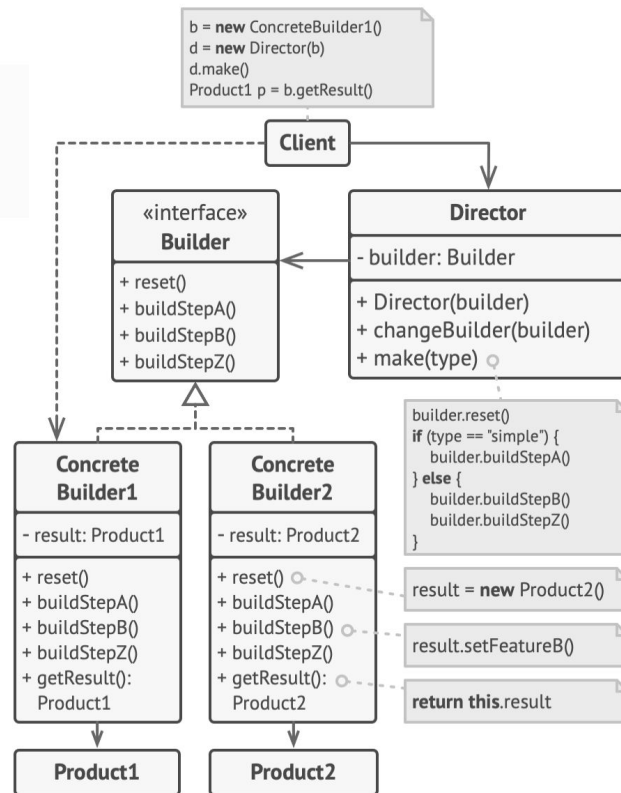
Estrutura do Padrão

- Builder (Interface) – define as etapas de construção.
- ConcreteBuilder – implementa as etapas específicas para construir um tipo de produto.
- Product – o objeto complexo resultante.
- Director (opcional) – define a ordem das etapas de construção.
- Client – escolhe o Builder e inicia o processo.
- O Diretor é opcional, mas útil para padronizar a construção e evitar repetição de código.

1 A interface **Builder** declara etapas de construção do produto que são comuns a todos os tipos de builders.

2 Builders Concretos provém diferentes implementações das etapas de construção. Builders concretos podem produzir produtos que não seguem a interface comum.

3 Produtos são os objetos resultantes. Produtos construídos por diferentes builders não precisam pertencer a mesma interface ou hierarquia da classe.



4 A classe **Diretor** define a ordem na qual as etapas de construção são chamadas, então você pode criar e reutilizar configurações específicas de produtos.

5 O **Cliente** deve associar um dos objetos builders com o diretor. Usualmente isso é feito apenas uma vez, através de parâmetros do construtor do diretor. O diretor então usa aquele objeto builder para todas as futuras construções. Contudo, há uma abordagem alternativa para quando o cliente passa o objeto builder ao método de produção do diretor. Nesse caso, você pode usar um builder diferente a cada vez que você produzir alguma coisa com o diretor.

```
// Produto
class Car(var seats: Int = 0, var engine: String = "", var gps: Boolean = false)

// Interface Builder
interface Builder {
    fun setSeats(num: Int)
    fun setEngine(type: String)
    fun setGPS(enabled: Boolean)
    fun getResult(): Car
}

// Builder concreto
class CarBuilder : Builder {
    private var car = Car()
    override fun setSeats(num: Int) { car.seats = num }
    override fun setEngine(type: String) { car.engine = type }
    override fun setGPS(enabled: Boolean) { car.gps = enabled }
    override fun getResult() = car
}

// Diretor
class Director {
    fun constructSportsCar(builder: Builder) {
        builder.setSeats(2)
        builder.setEngine("V8")
        builder.setGPS(true)
    }
}

// Cliente
fun main() {
    val builder = CarBuilder()
    val director = Director()
    director.constructSportsCar(builder)
    val car = builder.getResult()
    println("Carro esportivo: ${car.engine}, ${car.seats} lugares, GPS=${car.gps}")
}
```

Exemplo em Kotlin

```
# Produto
class Car:
    def __init__(self):
        self.seats = 0
        self.engine = ""
        self.gps = False

# Interface Builder
class Builder:
    def set_seats(self, num): pass
    def set_engine(self, type): pass
    def set_gps(self, enabled): pass
    def get_result(self): pass

# Builder concreto
class CarBuilder(Builder):
    def __init__(self):
        self.car = Car()

    def set_seats(self, num):
        self.car.seats = num

    def set_engine(self, type):
        self.car.engine = type

    def set_gps(self, enabled):
        self.car.gps = enabled

    def get_result(self):
        return self.car

# Diretor
class Director:
    def construct_sports_car(self, builder):
        builder.set_seats(2)
        builder.set_engine("V8")
        builder.set_gps(True)

# Cliente
if __name__ == "__main__":
    builder = CarBuilder()
    director = Director()
    director.construct_sports_car(builder)
    car = builder.get_result()
    print(f"Carro esportivo: motor {car.engine}, {car.seats} lugares, GPS={car.gps}")
```

Exemplo em Python

Aplicabilidade

- **Use o Builder quando:**
 - Um objeto complexo precisa ser criado em várias etapas ou combinações.
 - Deseja evitar construtores longos e confusos.
 - Precisa criar diferentes representações do mesmo produto (ex: casa simples, mansão, cabana).
 - O processo de construção precisa ser independente da forma final do produto.

Prós e Contras

- **Vantagens**

- Permite criar objetos complexos passo a passo.
- Facilita a criação de múltiplas representações do mesmo produto.
- Isola a lógica de construção da lógica de uso.
- Segue o Princípio da Responsabilidade Única.

- **Desvantagens**

- Aumenta a quantidade de classes no projeto.
- Pode parecer excessivo para objetos simples.



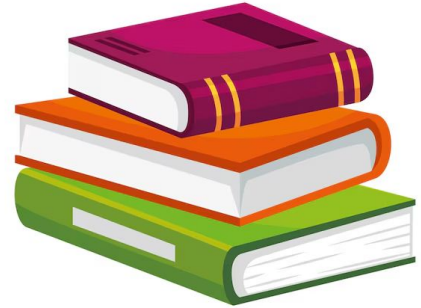
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado! Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

